BÁO CÁO THỰC HÀNH

# LAB 03
# DEBUGGING KERNEL & PHÂN TÍCH HIỆU NĂNG

Học phần: Embedded Linux Systems

**Mã số sinh viên:** SE203134
**Họ và tên:** [BỔ SUNG HỌ TÊN]
**Thời gian thực hành:** 22–26/10/2026
**Hệ thống:** Ubuntu 22.04 / QEMU ARMv7 Cortex-A9

Phiên bản báo cáo dựa trên log & ảnh bằng chứng đã cung cấp.
Các mục chưa có file gốc được đánh dấu để bổ sung trước khi nộp.

# TÓM TẮT KẾT QUẢ

LAB 03 khảo sát ba nhóm kỹ thuật quan trọng trong hệ thống Linux nhúng: **debugging từ xa**, **phân tích lỗi bộ nhớ và hiệu năng**, và **đánh giá độ trễ real-time**. Môi trường thử nghiệm gồm Ubuntu Host x86_64 và thiết bị ARMv7 Cortex-A9 giả lập bằng QEMU.

| Hạng mục | Kết quả ghi nhận | Bằng chứng chính |
|---|---|---|
| A — GDB | Đã đặt breakpoint và tìm ba bug: out-of-bounds, use-after-free, integer overflow. | GDB session + screenshot GDB mới |
| B — Valgrind | Trước: 16.900 B/250 blocks; sau: 0 B và 0 errors. | valgrind_report.txt / valgrind_fixed.txt |
| C — strace/perf | Host 129 syscall; ARM 101 syscall; perf 383 samples. | strace_summary.txt / perf_report.txt / c4 log |
| D — cyclictest | Max normal 293 µs; dưới tải 701 µs. | hist_standard.txt / hist_stress.txt / 2 PNG |

**Phạm vi xác nhận:** Đây là số liệu của các phiên được cung cấp. Việc đo PREEMPT_RT riêng **chưa được thực hiện**; phần so sánh RT được phân tích bằng cơ chế lý thuyết. File log GDB mới đã được người thực hành xác nhận qua screenshot nhưng chưa được tải lên archive nguồn của báo cáo.

## Mục lục

1. Môi trường và phương pháp
2. Part A — GDB Remote Debugging
3. Part B — Valgrind và memory analysis
4. Part C — System call & CPU profiling
5. Part D — Real-time latency
6. Trả lời 5 câu hỏi phân tích
7. Khó khăn, kết luận và tài liệu

Phụ lục: nguồn bằng chứng và checklist

# 1. MÔI TRƯỜNG VÀ PHƯƠNG PHÁP

| Thành phần | Thiết lập thực tế |
|---|---|
| Host OS | Ubuntu 22.04, x86_64 |
| Kernel Host | 6.8.0-138-generic, PREEMPT_DYNAMIC; CONFIG_PREEMPT_VOLUNTARY=y |
| Môi trường nhúng | QEMU system-arm, vexpress-a9, ARM Cortex-A9, RAM 512 MB |
| Kernel target | Linux 5.15.0 (LAB 01) và initramfs BusyBox (LAB 02/03) |
| Toolchain | arm-linux-gnueabihf-gcc; GDB multiarch / gdbserver (target) |
| Công cụ Host | Valgrind 3.18.1; strace; perf; cyclictest; gnuplot; stress-ng |

Quy trình: xây dựng/chạy chương trình trên Host hoặc QEMU tùy bài, thu log gốc, so sánh hành vi lỗi với trạng thái sửa lỗi, tổng hợp kết quả định lượng vào báo cáo. Các ảnh thuộc LAB 02 hoặc Assignment khác **không** được dùng làm bằng chứng LAB 03.

### Giới hạn đo lường

- Mọi tỷ lệ % của **strace -c** là tỷ lệ thời gian trong các system call mà strace thống kê, không phải tỷ lệ CPU của cả chương trình.
- VmRSS/VmSize là thống kê kernel về tiến trình; chúng không tự chứng minh có/không có memory leak. Kết luận về leak dựa vào Valgrind.
- Hai lần đo cyclictest chịu điều kiện lịch CPU khác nhau; đỉnh latency là quan sát của các lần chạy này, không phải cam kết worst-case tuyệt đối của kernel.

# 2. PART A — REMOTE DEBUGGING VỚI GDB

## 2.1. Mục tiêu, kiến trúc và thiết lập

Ứng dụng **buggy_app.c** được biên dịch ARM 32-bit với debug symbols (**-g -O0**) và nạp vào QEMU. Trên target, **gdbserver** lắng nghe TCP port 1234; Host dùng **gdb-multiarch** kết nối và ra lệnh đặt breakpoint, kiểm tra biến, backtrace và thanh ghi ARM.

```
Host x86_64: gdb-multiarch buggy_app
| GDB Remote Serial Protocol qua TCP:1234
QEMU ARMv7: gdbserver :1234 /usr/bin/buggy_app
```

```
minh@hayaminlinux:~$ cd ~/embedded_lab3/apps
minh@hayaminlinux:~/embedded_lab3/apps$ gdb-multiarch buggy_app
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from buggy_app...
(gdb)
```

*Hình A1 — GDB multiarch kết nối và đặt breakpoint với ARM QEMU.*

## 2.2. Ba lỗi xác định được

| Bug / vị trí | Quan sát từ GDB | Phân tích và phương án sửa |
|---|---|---|
| A. Array out-of-bounds **buggy_app.c:10** | **i=5**, **len=5**; mảng {10,20,30,40,50}; backtrace process_data  main. | Điều kiện vòng lặp là **i <= len**; sửa thành **i < len**, kiểm tra đầu vào và độ dài. |
| B. Use-after-free **buggy_app.c:22–24** | Trước free: buf = 0x413198 chứa “Hello Embedded Linux”; sau free, cùng địa chỉ nhưng nội dung không còn hợp lệ. | Hàm trả con trỏ đã free; chuyển quyền sở hữu cho caller và free sau khi dùng, hoặc trả bản copy độc lập. |
| C. Integer overflow **buggy_app.c:29–31** | a=b=100000; kết quả số học 10.000.000.000; giá trị int quan sát 1.410.065.408. | Dùng **int64_t/long long** và ép kiểu **trước khi nhân**; kiểm tra overflow trước phép toán. |

```
(gdb) disable 4
(gdb) disable 5
(gdb) break calculate
Breakpoint 6 at 0x400702: file buggy_app.c, line 29.
(gdb) continue
Continuing.

Breakpoint 6, calculate (a=100000, b=100000) at buggy_app.c:29
29          int result = a * b;
(gdb) info args
a = 100000
b = 100000
(gdb) list
24          return buf;
25      }
26
27      /* Bug 3: Integer overflow */
28      int calculate(int a, int b) {
29          int result = a * b;
30
31          printf("calculate(%d, %d) = %d\n",
32                 a, b, result);
33
(gdb) print (long long)a * (long long)b
$6 = 10000000000
(gdb) next
31          printf("calculate(%d, %d) = %d\n",
(gdb) print result
$7 = 1410065408
(gdb) backtrace
#0  calculate (a=100000, b=100000) at buggy_app.c:31
#1  0x004007b0 in main () at buggy_app.c:57
(gdb) info registers pc sp lr
pc             0x40070c            0x40070c <calculate+20>
sp             0x7efffcd8          0x7efffcd8
lr             0x4007b1            4196273
(gdb)
```

*Hình A2 — Breakpoint tại calculate(), so sánh giá trị 64-bit và int 32-bit.*




*Hình A3 — Bằng chứng mới: file gdb_session_full.txt ghi remote connection, bốn breakpoint và các lần backtrace.*

### 2.3. Thao tác debug và tính đầy đủ bằng chứng

Phiên GDB mới đã được thực hiện ngày 26/10/2026. Screenshot xác nhận **Remote debugging using 127.0.0.1:1234**, breakpoint ở các dòng 10, 22, 24, 29, cũng như các lệnh **print**, **backtrace**, **info registers pc sp lr**. Log cũ **gdb_session.txt** dài 68 dòng đã được lưu từ phiên trước. File **gdb_session_full.txt** hiện nằm trên Ubuntu của người thực hành, chưa có bản text trong archive đính kèm; cần chép vào bản nộp để đáp ứng yêu cầu “toàn bộ session log”.

# 3. PART B — MEMORY ANALYSIS VỚI VALGRIND

## 3.1. Ba nguồn memory leak được phát hiện

Chương trình **leaky_app.c** tạo 100 Record, mỗi Record có mảng scores và chuỗi name, sau đó tạo 50 buffer thông điệp. Bản lỗi giải phóng Record mà không free hai vùng nhớ con và bỏ quên buffer thông điệp.

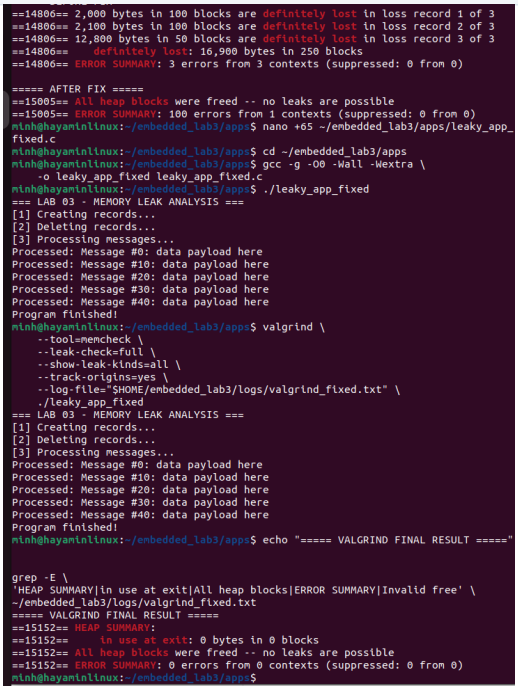
| Lỗi / allocation stack | Blocks | Definitely lost |
|---|---|---|
| scores: create_record(), leaky_app.c:43 | 100 | 2.000 bytes |
| name: create_record(), leaky_app.c:34 | 100 | 2.100 bytes |
| message buf: process_messages(), leaky_app.c:83 | 50 | 12.800 bytes |
| **Tổng** | **250** | **16.900 bytes** |

```
LEAK SUMMARY:
definitely lost: 16,900 bytes in 250 blocks
indirectly lost: 0 bytes in 0 blocks
ERROR SUMMARY: 3 errors from 3 contexts
```

## 3.2. Sửa và chạy lại

Phiên bản **leaky_app_fixed.c** bổ sung **free(r->name)**, **free(r->scores)** và **free(buf)**, đảm bảo chỉ free mỗi vùng đúng một lần. Ở lần sửa trung gian từng phát sinh 100 lỗi invalid free (double free); sau khi sửa lại, kết quả Valgrind cuối cùng không còn lỗi.

```
HEAP SUMMARY:
in use at exit: 0 bytes in 0 blocks
total heap usage: 352 allocs, 352 frees
All heap blocks were freed -- no leaks are possible
ERROR SUMMARY: 0 errors from 0 contexts
```




*Hình B1 — Kết quả Valgrind cuối cùng: 0 bytes in use, 0 errors.*

## 3.3. Quan sát bộ nhớ trên QEMU ARM

| Trường /proc/PID/status | Bản lỗi: trước/sau | Bản sửa: trước/sau |
|---|---|---|
| VmSize | 652 / 652 kB | 652 / 652 kB |
| VmRSS | 4 / 4 kB | 4 / 4 kB |
| VmData | 152 / 152 kB | 152 / 152 kB |
| VmStk | 132 / 132 kB | 132 / 132 kB |
| Exit code | 0 | 0 |

Cả hai bản đo được trong QEMU và có vùng nhớ **[heap]**, **[stack]**, **[vdso]** trong /proc/PID/maps. Không quan sát thấy biến thiên VmRSS qua hai thời điểm; vì leak chỉ ~16,9 KB và allocator có thể giữ lại trang nhớ, điều này không phủ định kết luận từ Valgrind. Phiên đo thủ công ban đầu bỏ lỡ tiến trình sau khi kết thúc; đã khắc phục bằng script đo tự động.

# 4. PART C — SYSTEM CALL VÀ CPU PROFILING

## 4.1. strace -c trên Ubuntu Host

| System call | % time | Số lần gọi |
|---|---|---|
| write | 46,05% | 76 |
| mmap | 21,47% | 14 |
| openat | 11,86% | 4 |
| mprotect | 7,77% | 5 |
| pread64 | 2,26% | 4 |
| **Tổng** | **100%** | **129 syscalls** |

Tổng thời gian trong syscall = **0,000708 giây**. **write()** được gọi 76 lần dù ứng dụng tạo 10.000 dòng bằng fprintf(), do I/O buffering. Thời gian chạy bản đo ban đầu: khoảng 23 ms, trong đó ~21 ms ở userspace. Như vậy syscall write chiếm tỷ lệ cao trong *system-call time* nhưng không chiếm đa số *total CPU time*.

## 4.2. perf: CPU hotspots và hardware counters

Do benchmark bản đầu quá ngắn, chương trình được build lại với **ITERATIONS=10000**. perf ghi nhận 383 CPU samples của sự kiện cpu-clock:u và không mất mẫu.

| Hàm / symbol | Overhead | Shared object |
|---|---|---|
| __ieee754_log_fma | 42,56% | libm.so.6 |
| heavy_compute | 42,30% | perf_target_profile |
| __log_finite@GLIBC_2.15@plt | 7,83% | libm.so.6 |
| log@plt | 7,31% | perf_target_profile |

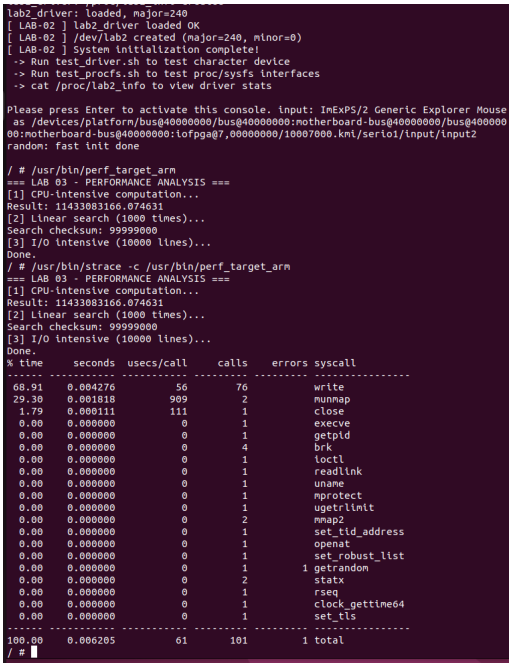
| Counter | Kết quả |
|---|---|
| CPU elapsed (perf stat) | 3,894387170 giây |
| CPU task-clock | 3.890,88 ms (~0,999 CPU) |
| User / System | 3,888377 s / 0,004002 s |
| Core cycles | 14.709.721.733 (time running ~99,49%) |
| Core instructions | 73.258.084.435 (~4,98 IPC) |
| Page faults | 371 |
| Context switches | 68 |

Kết quả là dấu hiệu **CPU-bound**: hàm heavy_compute() thực hiện sqrt() × log() lặp lại, các mẫu tập trung ở tính toán log và vòng lặp. Máy host có CPU hybrid; nhóm counter cpu_core/cpu_atom có tỷ lệ thời gian đo khác nhau, không cộng các giá trị trực tiếp.

### 4.3. strace trong QEMU ARM

| System call | % time ARM | Calls ARM |
|---|---|---|
| write | 68,91% | 76 |
| munmap | 29,30% | 2 |
| close | 1,79% | 1 |
| Các syscall còn lại | 0,00% do làm tròn | 22 |
| **Tổng** | **100%** | **101** |

Tổng syscall time trên ARM ~**0,006205 giây**. Ứng dụng ARM liên kết tĩnh, còn binary host liên kết động; hệ ABI, thư viện và môi trường giả lập cũng khác nhau. Do đó số lần gọi/tỷ lệ strace khác nhau **không** đủ để kết luận tốc độ CPU thực của ARM so với x86.




*Hình C1 — perf_target_arm và bảng strace -c trong QEMU ARM.*

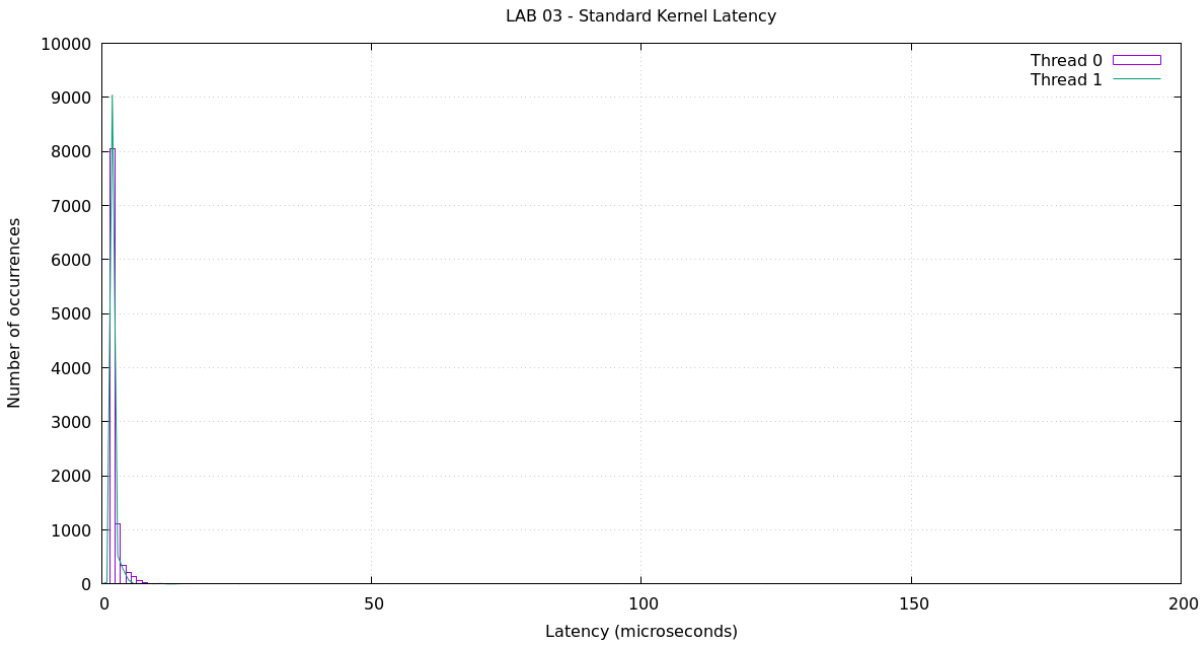
# 5. PART D — ĐỘ TRỄ REAL-TIME VỚI CYCLICTEST

### 5.1. Thiết lập và số liệu

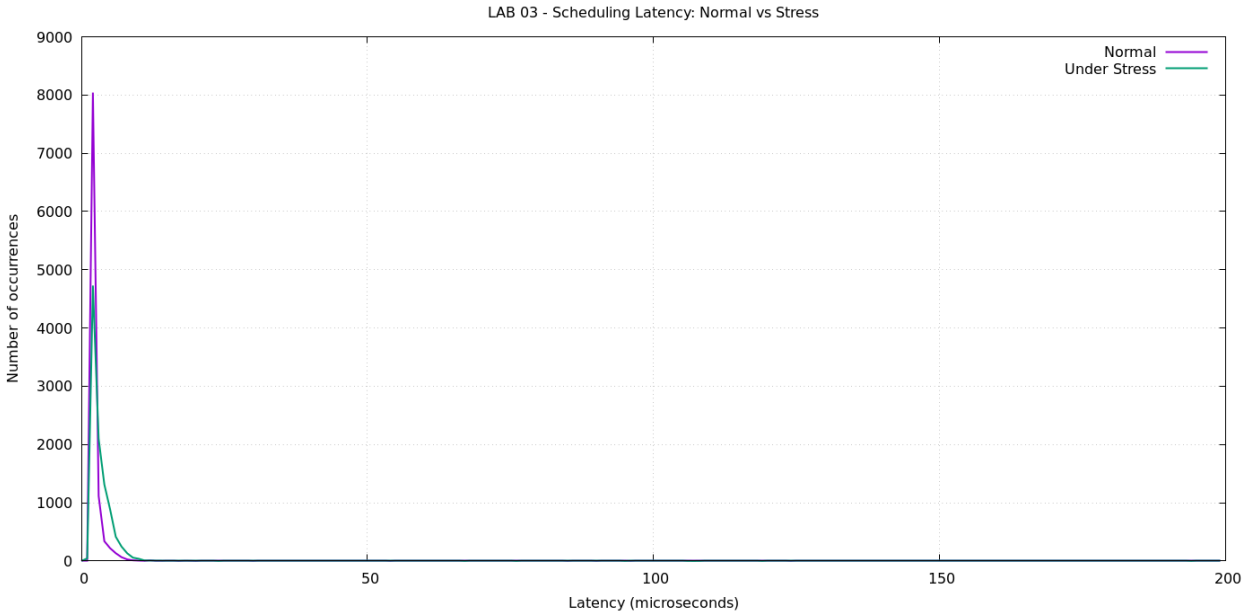
Phép đo chạy trên Ubuntu Host (không đo trên QEMU), với **4 threads**, ưu tiên **SCHED_FIFO 80**, chu kỳ cơ bản 1 ms, 10.000 loops/thread. Hai trạng thái gồm bình thường và chạy stress-ng tạo tải CPU, I/O, memory. Kernel host là **6.8.0-138-generic**, cấu hình **CONFIG_PREEMPT_VOLUNTARY=y** cùng **CONFIG_PREEMPT_DYNAMIC=y**; chưa xác nhận có kernel PREEMPT_RT.

| Thread | Min normal | Avg normal | Max normal | Min stress | Avg stress | Max stress |
|---|---|---|---|---|---|---|
| T0 | 2 | 2 | 124 | 1 | 3 | 194 |
| T1 | 1 | 2 | 16 | 1 | 3 | 110 |
| T2 | 2 | 2 | 257 | 1 | 3 | 701 |
| T3 | 1 | 2 | 293 | 1 | 3 | 107 |

Đơn vị: **microsecond (µs)**. Max cao nhất bình thường = **293 µs**; khi chịu tải = **701 µs** (tăng 408 µs, tương ứng khoảng **2,39 lần**). Avg 4 thread tăng 23 µs. Histogram overflows theo thread: normal **0,0,5,3**; stress **0,0,2,0**. Giá trị vượt 200 µs không hiển thị đầy đủ trên biểu đồ giới hạn 0–200 µs.




*Hình D1 — Histogram của Standard Kernel: phân phối latency thấp, đuôi dài.*




*Hình D2 — So sánh phân phối Thread 0: normal và stress (trục X đến 200 µs).*

### 5.2. Nhận xét real-time

Max cao hơn Avg nhiều lần cho thấy các sự kiện độ trễ đột biến. Một thiết bị robot yêu cầu phản hồi **nhỏ hơn 500 µs** sẽ không được chứng minh đáp ứng trong các điều kiện đã đo vì có mẫu **701 µs** dưới tải. Nên cân nhắc kernel PREEMPT_RT cùng cấu hình IRQ/CPU affinity, locking, page-fault avoidance và kiểm thử worst-case trên phần cứng đích. **Chưa có dữ liệu đo RT kernel thực tế**, nên không đưa ra bảng RT có số liệu tưởng tượng.

## 6. TRẢ LỜI 5 CÂU HỎI PHÂN TÍCH

### Câu 1. Chuỗi sự kiện khi gõ “next” trong GDB remote debugging

Khi gõ **next**, GDB chạy trên Host phân tích debug symbols và vị trí nguồn; tùy ngữ cảnh, nó tạo breakpoint tạm ở vị trí sẽ dừng sau “step over”, rồi gửi lệnh đến **gdbserver** theo **GDB Remote Serial Protocol (RSP)** qua TCP 127.0.0.1:1234. gdbserver sử dụng giao diện debug của Linux (chẳng hạn ptrace) để điều khiển tiến trình ARM trong QEMU, cho tiến trình chạy tiếp và dừng khi đạt điều kiện/breakpoint. Trạng thái dừng, địa chỉ PC và dữ liệu thanh ghi/bộ nhớ được trả theo gói RSP; GDB ánh xạ địa chỉ về dòng C rồi hiển thị biến và stack.

Lý do dùng **gdb-multiarch**: GDB trên Host phải hiểu target ARM EABI, tập thanh ghi, calling convention và debug symbols ARM; bản GDB cài mặc định cho x86_64 có thể không có đầy đủ target support. Thực tế trong LAB, GDB multiarch kết nối được cổng 1234 và hiển thị breakpoint ở buggy_app.c, ví dụ process_data(), calculate().

### Câu 2. Definitely lost và indirectly lost; ba lỗi bộ nhớ khác

**Definitely lost** là các block heap không còn con trỏ hợp lệ bắt đầu từ root set (stack, registers, globals), nên Valgrind xác nhận không thể giải phóng bằng những con trỏ đang có. **Indirectly lost** là block chỉ có thể truy cập qua một block khác đã mất; ví dụ một cấu trúc ngoài mất con trỏ, khiến các con trỏ đến cấp phát con cũng mất theo. Trong mã **delete_record()**, vì **free(r)** được gọi trước mà **r->name** và **r->scores** không được free, hai con trỏ bên trong biến mất: log thực tế ghi **definitely lost** 2.100 bytes cho name và 2.000 bytes cho scores (không phải indirectly lost).

Ngoài leak, Valgrind Memcheck phát hiện: (1) invalid read/write hoặc out-of-bounds, (2) double free/invalid free, (3) sử dụng giá trị chưa khởi tạo và (4) đọc/ghi vào vùng đã giải phóng. Chính phiên sửa trung gian của LAB từng xuất hiện **100 invalid frees** trước khi sửa hoàn toàn; kết quả cuối 0 errors.

## Câu 3. Hai phương án tối ưu heavy_compute()

**Phương án A — loại phép tính lặp dư thừa (algorithmic hoisting/precomputation):** Dữ liệu arr không đổi giữa các vòng ITERATIONS, nên có thể tính một lượt **S = Σsqrt(arr[i])·log(arr[i]+1)** rồi tái sử dụng S ở các vòng thay vì tính N phép sqrt/log mỗi lần. Độ phức tạp phần toán học từ O(ITERATIONS×N) về gần O(N+ITERATIONS); lý thuyết có thể giảm hàng nghìn lần số lần gọi libm khi ITERATIONS=10000, nhưng cần kiểm thử sai khác dấu phẩy động do thay đổi thứ tự cộng. *Đây là dự đoán thuật toán, chưa benchmark thực nghiệm.*

**Phương án B — vector hóa và tối ưu vòng lặp/cache:** dùng -O3, unroll loop và dữ liệu truy cập liên tiếp; nếu yêu cầu độ chính xác cho phép đổi từ double sang float, dùng NEON ARMv7 cho những đoạn số học/vector. Chú ý ARMv7 NEON thông dụng không vector hóa trực tiếp hàm log(double): muốn tăng tốc phải thay bằng xấp xỉ vector được xác thực sai số. Một mức cải thiện ~1,1–2× cho phần có thể tối ưu là mục tiêu thử nghiệm, không phải kết quả đã đo. Benchmark từng thay đổi để kiểm tra tốc độ, sai số và điều kiện cạnh.

Căn cứ: perf đã ghi 383 mẫu với __ieee754_log_fma ~42,56% và heavy_compute ~42,30%, nên ưu tiên giảm phép tính log/sqrt và chi phí vòng lặp thay vì tối ưu write().

## Câu 4. Vì sao có latency spikes? PREEMPT_RT xử lý thế nào?

- **Interrupt handling:** đoạn xử lý hard IRQ và vùng tắt ngắt có thể trì hoãn thread ưu tiên cao. PREEMPT_RT thread hóa phần lớn interrupt handlers để có thể lập lịch/đặt ưu tiên và preempt nhiều công việc trong IRQ hơn (còn phần hard IRQ tối thiểu không thể loại bỏ hết).
- **Scheduler và non-preemptible critical sections:** standard kernel có đoạn giữ spinlock/tắt preemption khiến thread quan trọng phải chờ. PREEMPT_RT chuyển nhiều khóa thông thường thành rtmutex có priority inheritance, giảm priority inversion và mở rộng khả năng preempt; một số vùng khóa ngắn vẫn là raw spinlock.
- **Memory management và page faults:** page faults, memory reclaim, allocator contention hoặc I/O do paging khiến thread bị chậm. PREEMPT_RT giúp giảm thời gian khóa một số đường kernel nhưng **không loại bỏ page faults**; ứng dụng vẫn cần mlockall(), prefault stack/heap và không cấp phát động trong critical real-time loop.

Đối chiếu số liệu: Avg chỉ 2–3 µs nhưng Max đạt 293 µs bình thường, 701 µs khi stress; các spike có thể chịu tác động của đồng thời nhiều nguồn và dữ liệu hiện tại không chỉ đích danh IRQ/scheduler nào gây ra một mẫu cụ thể. Cần thêm ftrace/trace-cmd để quy nguyên nhân.

### Câu 5. Chọn công cụ theo tình huống dự án thực tế

| Hiện tượng | Công cụ ưu tiên và lý do |
|---|---|
| (a) Crash ngẫu nhiên sau vài giờ | **GDB** với core dump/gdbserver để xem backtrace, trạng thái thread, biến, địa chỉ lỗi; kết hợp Valgrind hoặc ASan nếu nghi hỏng bộ nhớ. |
| (b) Chạy chậm dần theo thời gian | **Valgrind** để tìm heap leaks; theo dõi VmRSS/process resources. Nếu không phải leak, dùng **strace** để tìm I/O/syscall chờ lâu. |
| (c) Thỉnh thoảng miss deadline | **cyclictest** để đo phân phối/worst-case scheduling latency (sau đó ftrace nếu cần xác định nguyên nhân). |
| (d) Một hàm làm hệ thống chậm | **perf** để lấy CPU samples/call graph và xem phần CPU overhead theo function. |

Áp dụng vào LAB: GDB tìm ba bug ở Part A; Valgrind phát hiện 16.900 bytes leak ở Part B; perf chỉ ra heavy_compute/libm ở Part C; cyclictest phát hiện spike 701 µs dưới tải ở Part D.

# 7. KHÓ KHĂN — KẾT LUẬN — TÀI LIỆU

## 7.1. Khó khăn và cách khắc phục

| Khó khăn thực tế | Cách xử lý đã thực hiện |
|---|---|
| Thiếu gdbserver ARM phù hợp, cần đúng shared libraries/architecture. | Lấy gdbserver armhf, kiểm tra ELF bằng file, bổ sung ARM dynamic loader/libs vào rootfs, dùng gdb-multiarch và sysroot. |
| GDB session lần đầu thiếu đầu phiên log. | Ghi phiên mới từ lúc remote connection, chụp screenshot và lưu gdb_session_full.txt trên Host. |
| Sửa leak nhưng phát sinh 100 invalid free. | Xem allocation/free stack trace bằng Valgrind, bỏ double free, build lại và đo đến 0 errors. |
| perf báo I/O permission denied, thời gian chạy quá ngắn. | Chuyển file output sang /tmp/perf_test_PID.txt; tăng ITERATIONS=10000; perf thu 383 samples. |
| Đo AFTER VmRSS bị mất vì tiến trình kết thúc sớm. | Dùng chương trình probe với pause và script đo tự động trong QEMU. |

## 7.2. Kết luận

Bài lab đã vận dụng đầy đủ chu trình debugging và quan sát hiệu năng trên Linux Host + ARM QEMU. GDB cho phép xác định điều kiện out-of-bounds, dùng lại vùng nhớ đã giải phóng và phép nhân tràn 32-bit. Valgrind cho thấy khả năng phát hiện leak và xác minh sau sửa. strace/perf làm rõ sự khác nhau giữa thời gian trong syscall và bottleneck xử lý CPU. cyclictest chứng minh average rất nhỏ vẫn có thể kèm peak latency lớn, nhất là khi tạo tải.

Phần thực hành A–D được chứng minh bởi các log/ảnh đã cung cấp. Trước khi nộp chính thức cần bổ sung bản gốc **gdb_session_full.txt**, binary ARM/x86_64, initramfs có thể boot và điền họ tên trên trang bìa.

## 7.3. Tài liệu và bằng chứng

- LAB03_EmbeddedLinux.docx — đề thực hành chính thức, mục IV–VI, 27 trang.

- log gốc: gdb_session.txt; valgrind_report.txt; valgrind_fixed.txt; strace_summary.txt; perf_report.txt; perf_stat.txt; hist_standard.txt; hist_stress.txt.
- Bằng chứng QEMU: b5_qemu_session.txt, c4_qemu_session.txt, screenshot ARM strace và GDB session mới (26/10/2026).
- Ảnh dữ liệu: latency_histogram.png, latency_comparison.png.
- GDB Remote Protocol; Valgrind Memcheck Manual; Linux perf; cyclictest/rt-tests; PREEMPT_RT documentation (các tài liệu tham khảo theo đề).

# PHỤ LỤC — KIỂM TRA TRƯỚC KHI NỘP

| Hạng mục | Trạng thái tại thời điểm tạo PDF |
|---|---|
| Báo cáo PDF + các hình | ĐÃ TẠO trong file này |
| 4 source C và log gốc A–D | Đã nhận trong evidence archive |
| Screenshot GDB mới | Có screenshot xác nhận; cần copy bản txt đầy đủ trên Ubuntu |
| buggy_app (ARM), leaky_app, leaky_app_fixed binaries | CHƯA CÓ trong archive, lấy từ ~/embedded_lab3/apps/ |
| initramfs_lab3.cpio.gz | CHƯA CÓ trong archive, cần kiểm tra và copy từ output/ |
| Họ và tên đầy đủ | CẦN ĐIỀN trước khi nộp |
| GitHub upload | CHƯA ĐƯỢC XÁC NHẬN trên remote main |

Tên file theo đề: **MSSV_HoTen_Lab03.tar.gz** chứa apps/, output/, logs/ và bao_cao/. Khi chuẩn bị GitHub, giữ thư mục **SE203134_Lab03** và các nhánh 01_Source, 02_Evidence, 03_Screenshots như bản phân loại; không trộn những ảnh thuộc bài khác vào thư mục bằng chứng của LAB 03.